



Data Abstractions

Dr. Obed Noé Sámano Abonce



TOPIC 1 · DATA ABSTRACTION

1.1 Data abstraction

Thinking about the what before the 'how'

Abstraction

What is abstraction?

To abstract is to preserve what is essential while hiding implementation details that clients do not need to use the abstraction.

Analogy: driving a car

- You use the wheel, pedals and gearshift.
- You don't need to know how fuel injection works.
- The interface (controls) is enough; the engine stays hidden.

In programming

Abstraction = managing complexity by separating **what** a component or data type does from **how** it is implemented inside.

INTERFACE → **what the user sees and uses**

IMPLEMENTATION → **the internal details**

Good software hides details behind simple interfaces.

Data abstraction

What is exposed and what is hidden

EXPOSED · Interface

- Which operations exist (add, search, remove...).
- What each operation takes in and returns.
- The expected meaning / behavior.
- It's the 'contract' with whoever uses it.

HIDDEN · Implementation

- How data is stored (array, list, tree...).
- The internal algorithms of each operation.
- Helper variables and memory management.
- Can change without affecting whoever uses it.

Data abstraction

The important idea

WHAT

The behavior

"I need to store values and always take out the last one I put in."



HOW

The implementation

"I'll do it with an array... or linked nodes... or by reusing a vector."

First, we define the WHAT the code must do. Then choose HOW to represent and implement it.

TOPIC 1 · DATA ABSTRACTION

1.2 Abstract Data Types (ADT)

An ADT formalizes the WHAT: values and operations without specifying the implementation.

ADT

What is an ADT?

An ADT is a model of a data type defined by its possible **values** and **operations** on them — independent of how it is implemented.

Values

- What information it represents and what valid states it has.
- E.g.: a Point represents a position in the plane, given by two coordinates.

Operations

- What can be done and what each operation guarantees.
- E.g.: in a Point, `distanceTo(p)` returns the straight-line distance to another point.

ADTs

Primitive types vs ADTs

Concept	What it is	Examples
Primitive data type	Comes built into the language. Its values and operations were already decided for you.	<code>int, double, char, bool</code>
ADT	You define it. You decide what values it holds and what operations it offers.	<code>Point, Student, Date, Queue, Stack</code>

ADTs

Example: the Stack

Behavior: LIFO

Last In, First Out: the last element to go in is the first to come out (like a stack of plates).

push (x)

places x on top

pop ()

removes the top element

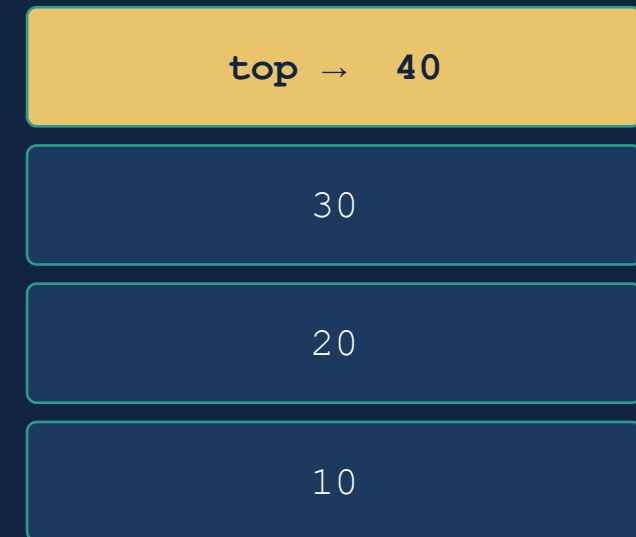
top ()

reads the top without removing it

empty ()

are there no elements?

Notice: we did NOT say how it's stored inside.



push(50) would put 50 on top; pop() would remove 40.

ADTs

From ADT to C++

```
// Stack ADT: the class separates WHAT from HOW
class Stack {
public:                // interface (the 'what')
    void push(int x); // push
    void pop();       // pop
    int  top() const; // read the top
    bool empty() const; // is it empty?

private:             // hidden implementation
    // ...array, list or vector...
    // (the 'how', invisible to the user)
};
```

Encapsulation

public: is the contract others see.

private: protects the details: no one touches them from outside.

Changing the private part does NOT break the code that uses the Stack.

TOPIC 1 · DATA ABSTRACTION

1.3 Levels of abstraction

The same idea looks different depending on which layer you view it from.

Levels of Abstraction

Levels

Application level (client)

Uses the ADT through its public interface. Depends on the contract, not the representation.

Logical level (abstract)

Defines the possible values and operations.

Implementation level

Uses data structures and algorithms to fulfil the contract.

Top to bottom, you go from 'what I need' to 'how I build it'.

Levels of Abstraction

Advantages

Information hiding: each level exposes the minimum needed and hides the rest. In C++ we achieve it with `public` / `private` in classes.



Fewer bugs

No one accidentally manipulates the internal data.



Safe changes

You rewrite the implementation without touching users.



Reusable code

The ADT serves many different programs.



Teamwork

Everyone programs against a clear interface.



TOPIC 1 · DATA ABSTRACTION

1.4 Templates in C++

The tool that lets us write data structures for ANY type.

Templates in C++

The problem: repeated code

```
// A different 'max' for each type... tedious
int    maxInt (int a, int b)
    { return a > b ? a : b; }

double maxDb1 (double a, double b)
    { return a > b ? a : b; }

char   maxChar(char a, char b)
    { return a > b ? a : b; }

// and string? and your own classes? ...
```

What do you notice?

- The logic is identical.
- Only the type changes.
- Copy-paste = bugs and costly maintenance.

Templates in C++

Solution

A **template** leaves the **type as a parameter**. You write the code ONCE and the compiler generates a version for each type you use.

1

You write 1 template

```
template <typename T>
```

2

You use it with a type

```
maximum(3, 7)
```

3

The compiler generates

```
an int version
```

Templates in C++

Function template

One algorithm, any type

```
template <typename T>    // T = any type
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

template <typename T>
void swapVals(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
```

The compiler deduces T

```
maximum(3, 7);
// T = int
maximum(2.5, 1.0);
// T = double
maximum('a', 'z');
// T = char
```

Templates in C++

Class templates: generic containers

Function templates: The compiler usually deduced the type

Class templates: You usually specify the type explicitly.

```
template <typename T>
class Box {
private:
    T content;
public:
    Box(T c) : content(c) {}
    T get() const { return content; }
    void set(T c) { content = c; }
};
```

Instantiated with <type>

```
Box<int>      bi(5);
Box<string>   bs("hi");
```

Templates in C++

Best practices

⚠ Keep template definitions visible

Keep both the signature and the body together in the header. Do not place template definitions in a separate `.cpp`. The compiler must generate the code at the point of use, and it can only see the header.

⚠ The type must support the operation

`maximum<T>` requires operator `>` ; if `T` is your own class, you must define that operator.

✓ typename vs class

Equivalent when declaring the parameter. Pick one and be consistent (typename is suggested).

✓ Name the parameter well

`T` is fine for generic types; use clear names when there are several (Key, Value).

Summary

1.1

Abstraction

Separate the what from the how.

1.2

ADT

Formalize the what: values + operations.

1.3

Levels

Application · logical · implementation.

1.4

Templates

Implement the how for any type.

Takeaway: in the course we will be designing ADTs and choosing the data structure that best implements them.

Next session: Recursion — functions that call themselves.

Exercise

Design and implementation of an ADT

A) Design an ADT for representing and manipulating points in the Cartesian plane. Each point is an abstract value.

- a) Domain:** the possible values represented by the ADT.
- b) Abstract state:** the information that describes each point.
- c) Operations:** the actions available to the client.

B) Implement the ADT in C++

Bibliography

- M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.
- V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas Gurrión, *Estructuras de datos y algoritmos fundamentales*. Monterrey, México: Editorial Digital del Tecnológico de Monterrey, 2020.